

Examen de Sistemas Inteligentes

Nombre y apellidos: _____

Fecha: _____

Pregunta 1 (traza)

PREGUNTA DE EXAMEN: TRAZA DE EJECUCIÓN

Enunciado

Considera la siguiente traza de ejecución. Se dispone de una clase `Nodo` con métodos `getCasilla()` y `getG()`. Se ha inicializado:

```
nodo1 = Nodo(casilla=(0,0), g=5)
nodo2 = Nodo(casilla=(1,2), g=8)
nodo3 = Nodo(casilla=(0,0), g=12)
heap = [nodo1, nodo2, nodo3]
g_antigua = [0]
```

Se ejecuta la siguiente llamada:

```
resultado = existe_nodo(heap, casilla=(0,0), g_antigua)
```

Responde a las siguientes cuestiones:

1. ¿Cuál es el valor de resultado después de la ejecución?
2. ¿Cuál es el valor de `g_antigua[0]` después de la ejecución?
3. Justifica por qué la función retorna en el punto que lo hace, sin examinar los demás elementos de heap.

Código (`existe_nodo`):

```
def existe_nodo(heap, casilla, g_antigua):
    for nodo in heap:
        if nodo.getCasilla() == casilla:
            g_antigua[0] = nodo.getG()
            return True
    return False
```

Pregunta 2 (test)

PREGUNTA DE EXAMEN: Sistemas Inteligentes (3º Grado)

Enunciado:

Consideremos una función `modifica_nodo` que busca actualizar un nodo dentro de una estructura heap utilizada en algoritmos de búsqueda como A*. La función recibe un heap, una casilla identificadora, nuevos valores de coste (`nueva_g`), nodo padre (actual) y una nueva calificación (`nueva_cal`).

Examen de Sistemas Inteligentes

Supongamos que ejecutamos las siguientes operaciones:

```
heap = [NodoA(casilla=5, g=10, cal=15),  
        NodoB(casilla=3, g=8, cal=12)]  
modifica_nodo(heap, 5, 20, NodoC, 25)  
modifica_nodo(heap, 7, 5, NodoD, 10)
```

¿Cuál será el resultado más probable después de estas dos llamadas?

A) El heap mantiene 2 nodos. NodoA tendrá g=20 y cal=25. NodoB permanecerá sin cambios. La función devuelve True la primera vez y False la segunda.

B) El heap mantiene 2 nodos. NodoA tendrá g=20 y cal=25. El heap se reorganiza dos veces. La función devuelve True dos veces.

C) El heap se reduce a 1 nodo porque modifica_nodo elimina nodos no encontrados. La función devuelve True la primera vez y genera excepción la segunda.

D) NodoA se duplica en el heap con los nuevos valores. El heap contiene 3 nodos. La función devuelve True dos veces.

Código (modifica_nodo):

```
def modifica_nodo(heap, casilla, nueva_g, actual, nueva_cal):  
    for nodo in heap:  
        if nodo.getCasilla() == casilla:  
            nodo.setG(nueva_g)  
            nodo.setPadre(actual)  
            nodo.cal = nueva_cal  
            heapq.heapify(heap)  
            return True  
    return False
```

Pregunta 3 (traza)

PREGUNTA DE EXAMEN: Traza de Ejecución de reconstruirCamino

Enunciado

Considera la siguiente estructura de datos para representar nodos en un grafo de búsqueda de caminos:

```
class Casilla:  
    def __init__(self, fila, col):  
        self._fila = fila  
        self._col = col  
    def getFila(self):  
        return self._fila  
    def getCol(self):
```

Examen de Sistemas Inteligentes

```
return self._col
```

class Nodo:

```
def __init__(self, casilla, padre=None):
    self.casilla = casilla
    self.padre = padre
def getCasilla(self):
    return self.casilla
```

Se proporciona la función `reconstruirCamino(nodo_final, camino)` que procesa un nodo final de un árbol de búsqueda para marcar el camino recorrido.

Situación concreta: Se ejecuta el siguiente código:

Crear una estructura de árbol de búsqueda (camino hacia atrás)

```
c0 = Casilla(0, 0)
c1 = Casilla(1, 1)
c2 = Casilla(2, 2)
c3 = Casilla(3, 1)
```

```
nodo0 = Nodo(c0, None)
nodo1 = Nodo(c1, nodo0)
nodo2 = Nodo(c2, nodo1)
nodo3 = Nodo(c3, nodo2)
```

Inicializar matriz 4x4 con guiones

```
matriz = [['-' for _ in range(4)] for _ in range(4)]
```

Llamada a la función

```
reconstruirCamino(nodo3, matriz)
```

Pregunta: ¿Cuál es el contenido de `matriz[3][1]`, `matriz[2][2]` y `matriz[1][1]` después de ejecutar `reconstruirCamino(nodo3, matriz)`? Explica qué casillas quedaron marcadas y por qué.

Código (`reconstruirCamino`):

```
def reconstruirCamino(nodo_final, camino):
    while nodo_final is not None:
        camino[nodo_final.getCasilla().getFila()][nodo_final.getCasilla().getCol()] = 'X'
        nodo_final = nodo_final.padre
```

Pregunta 4 (traza)

PREGUNTA DE EXAMEN: Traza de Ejecución - `printNodosExplorados`

Enunciado

Examen de Sistemas Inteligentes

Dado el siguiente fragmento de código que simula la representación de nodos explorados en una matriz:

```
def printNodosExplorados(alto, ancho, lista_interior):
    matriz = []
    for i in range(alto):
        matriz.append([])
        for j in range(ancho):
            matriz[i].append(-1)

    for i in range(len(lista_interior)):
        casillaNodo = lista_interior[i].casilla
        matriz[casillaNodo.fila][casillaNodo.col] = i

    for fila in matriz:
        print(' '.join(f"{num:<3}" for num in fila))
```

Supón que tienes las siguientes clases y datos:

```
class Casilla:
    def __init__(self, fila, col):
        self.fila = fila
        self.col = col

class Nodo:
    def __init__(self, casilla):
        self.casilla = casilla
```

Datos de entrada

```
nodos = [
    Nodo(Casilla(0, 1)),
    Nodo(Casilla(1, 0)),
    Nodo(Casilla(1, 2)),
    Nodo(Casilla(2, 1))
]
```

```
printNodosExplorados(3, 3, nodos)
```

Pregunta: ¿Cuál será la salida impresa exacta por pantalla? Describe el contenido de cada fila y el espaciado resultante.